UNIVERSIDADE DE LISBOA

FACULDADE DE CIÊNCIAS

DEPARTAMENTO DE INFORMÁTICA

Título do Trabalho

Nome Completo do Candidato

Mestrado em Designação do Mestrado
Especialização em Designação da Especialização / Perfil, se aplicável

Versão Provisória

Dissertação orientada por:
Profª. Doutora Nome Completo do Orientador
Prof. Doutor Nome Completo do Orientador

2025

Colocar aqui a Dedicatória.

Agradecimentos

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Resumo

Os resumos, em Português e em Inglês, são obrigatórios. O número de palavras que cada um deve ter depende da língua em que o trabalho é escrito. Para as regras que definem estas dimensões, consultar as normas de escrita do trabalho final no site da Faculdade de Ciências (`https://ciencias.ulisboa.pt/`).

Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue duis dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum. Typi non habent claritatem insitam; est usus legentis in iis qui facit eorum claritatem. Investigationes demonstraverunt lectores legere me lius quod ii legunt saepius. Claritas est etiam processus dynamicus, qui sequitur mutationem consuetudium lectorum. Mirum est notare quam littera gothica, quam nunc putamus parum claram, anteposuerit litterarum formas humanitatis per seacula quarta decima et quinta decima. Eodem modo typi, qui nunc nobis videntur parum clari, fiant sollemnes in futurum. Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue duis dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum. Typi non habent claritatem insitam; est usus legentis in iis qui facit eorum claritatem. Investigationes demonstraverunt lectores legere me lius quod ii legunt saepius.

Palavras-chave: um máximo de 5 palavras-chave

Abstract

The abstract should have at most 300 words.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue duis dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum. Typi non habent claritatem insitam; est usus legentis in iis qui facit eorum claritatem. Investigationes demonstraverunt lectores legere me lius quod ii legunt saepius. Claritas est etiam processus dynamicus, qui sequitur mutationem consuetudium lectorum. Mirum est notare quam littera gothica, quam nunc putamus parum claram, anteposuerit litterarum formas humanitatis per seacula quarta decima et quinta decima. Eodem modo typi, qui nunc nobis videntur parum clari, fiant sollemnes in futurum. Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue duis dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum. Typi non habent claritatem insitam; est usus legentis in iis qui facit eorum claritatem.

Keywords: a maximum of 5 keywords (in English)

Índice

Lista de Figuras **xiii**

Lista de Tabelas **xv**

1 Introdução **1**

1.1 Contexto e Motivação 1

1.2 Objetivos 1

1.3 Contribuições e Resultados 1

1.4 Estrutura da Dissertação 1

2 Estado da Arte **3**

2.1 Enquadramento Teórico 3

2.1.1 Geração automática de código 3

2.1.2 Ciclo de Vida do Desenvolvimento de Software 4

2.1.3 Limitações dos LLMs e Técnicas de Refinamento 5

2.1.4 Ferramentas de Orquestração de Fluxos 6

2.2 Trabalho Relacionado 7

2.2.1 Geração Automática de Código 7

2.2.2 Engenharia de Prompts 10

3 Visão Geral e Abordagem à Solução **13**

3.1 Arquitetura e Design da Solução 13

3.2 Fases de Desenvolvimento 13

3.3 Desafios e Decisões de Implementação 13

4 Desenvolvimento Técnico da Solução **15**

4.1 Desenvolvimento Inicial com Prompts Manuais 15

4.2 Fluxo de Automação em n8n 15

4.3 Iterações e Refinamentos 15

5 Testes e Metodologia de Avaliação **17**

5.1 Plano Experimental 17

5.2 Métricas e Critérios de Avaliação 17

6 Resultados e Discussão **19**
6.1 Análise Comparativa 19
6.2 Discussão dos Resultados 19

7 Conclusão **21**
7.1 Conclusões Finais 21
7.2 Trabalho Futuro 21

A Material de Apoio **23**
A.1 Prompts Utilizados 23
A.2 Resultados Detalhados 23

Bibliografia **25**

Lista de Figuras

2.1 Comparação entre os modelos Waterfall e modelos Iterativos 5
2.2 Pipeline de especialização do Code Llama[9]. 7
2.3 Linha temporal de alguns dos modelos e ferramentas de geração de código analisados nesta secção. 9

Lista de Tabelas

Capítulo 1

Introdução

1.1 Contexto e Motivação

1.2 Objetivos

1.3 Contribuições e Resultados

1.4 Estrutura da Dissertação

Este documento está organizado da seguinte forma:

- Capítulo 2 – Estado da Arte.
- Capítulo 3 – Visão Geral e Abordagem à Solução.
- Capítulo 4 – Desenvolvimento Técnico da Solução.
- Capítulo 5 – Testes e metodologia de avaliação.
- Capítulo 6 – Resultados e discussão.
- Capítulo 7 – Conclusão e trabalho futuro.

Capítulo 2

Estado da Arte

2.1 Enquadramento Teórico

Esta secção serve como base teórica para clarificar alguns conceitos importantes para o projeto, permitindo uma melhor compreensão das secções que se seguem.

2.1.1 Geração automática de código

A geração automática de código consiste na produção de código fonte a partir de descrições de alto nível, como requisitos em linguagem natural, modelos formais ou representações estruturadas, com o objetivo de acelerar o desenvolvimento e reduzir tarefas repetitivas.

Apesar da geração automática de código estar hoje em dia associada à inteligência artificial generativa, a ideia não é nova. Abordagens como o Model-Driven Engineering (MDE)[28, 4] e as linguagens específicas de domínio (DSLs)[23] já permitiam gerar código a partir de modelos formais ou descrições declarativas. No entanto, estas técnicas assentam em regras rígidas e gramáticas estritamente definidas, sendo eficazes sobretudo em domínios controlados e pouco flexíveis perante requisitos abertos ou em linguagem natural.

Os modelos de IA generativa, amplamente disponibilizados a partir de 2022, marcaram uma evolução na geração automática de código ao permitirem interpretar instruções pouco estruturadas em linguagem natural[11]. Esta capacidade resulta do treino em grandes volumes de dados heterogéneos, que combinam linguagem natural e código fonte, permitindo aprender padrões sintáticos e relações entre componentes de programas.

Uma parte importante desta evolução deve-se à arquitetura *Transformer*, proposta em 2017 por Vaswani et al.[31]. Os *Transformers* utilizam mecanismos de atenção que relacionam tokens distantes numa sequência, capturando dependências de longo alcance e identificando as partes mais relevantes da instrução, o que se revelou altamente eficiente para tarefas de linguagem natural e permitiu treinar modelos de grande escala capazes de gerar código coerente[11].

Nos últimos anos têm surgido diversos sistemas que exploram modelos de linguagem para apoiar ou automatizar partes do desenvolvimento de software, recorrendo a modelos únicos ou a combinações de etapas especializadas para orientar e validar a geração. Esta diversidade reflete a maturidade crescente da área e o interesse em integrar LLMs como assistentes ativos no fluxo de trabalho do programador. Exemplos representativos são discutidos na secção 2.2.1.

2.1.2 Ciclo de Vida do Desenvolvimento de Software

Apesar do potencial das LLMs na geração de código, a construção de um produto de software envolve um conjunto mais amplo de fases, cuja natureza e formalização variam consoante o contexto organizacional. Para contextualizar o papel da automatização, é necessário compreender o ciclo de vida do desenvolvimento de software[30] e identificar as etapas onde esta pode ser introduzida.

Estas etapas incluem, em primeiro lugar, o levantamento e análise de requisitos, que visam garantir que as necessidades do cliente sejam adequadamente compreendidas, estruturadas e documentadas. O processo de levantamento de requisitos envolve diversas técnicas, incluindo entrevistas e revisões literárias, como evidenciado por Lim et al.[18], que destacam a importância de métodos eficazes para alcançar uma compreensão abrangente das necessidades dos utilizadores, uma vez que os requisitos devidamente capturados constituem a base sobre a qual todo o ciclo de vida do software é construído.

Com os requisitos definidos, a modelação funcional e estrutural do sistema surge como etapa subsequente, frequentemente suportada por linguagens formais e semi-formais de representação como a UML (Unified Modeling Language), BPMN (Business Process Model and Notation) para processos de negócio ou SysML (Systems Modeling Language) para sistemas complexos. Estas linguagens permitem traduzir requisitos textuais em representações visuais, facilitando a análise, validação e comunicação entre equipas técnicas e não técnicas. Os modelos UML desempenham um papel crítico na documentação e validação de requisitos, servindo como elementos centrais de coordenação entre diferentes perspetivas do sistema[3]. Esta modelação inclui diagramas de casos de uso, classes, atividades ou sequência, que complementam a descrição funcional do sistema e ajudam a desenhar o comportamento desejado. Neste contexto, a conceção de interfaces e fluxos de interação emerge como um artefacto fundamental, permitindo validar o comportamento esperado do sistema do ponto de vista do utilizador e servindo de base para decisões técnicas posteriores, nomeadamente ao nível da definição de APIs e da implementação.

A definição de APIs (Application Programming Interfaces) e dos componentes arquiteturais constitui outra etapa fundamental, pois estabelece os contratos de comunicação entre as diferentes partes do sistema. Especificações claras de APIs permitem uma implementação mais consistente, facilitam a integração entre módulos e suportam estratégias de teste orientadas a componentes e serviços.

Depois da definição dos componentes e das interfaces do sistema, segue-se a fase de implementação,

onde o código é desenvolvido de acordo com as especificações definidas. Esta etapa envolve não apenas a programação propriamente dita, mas também a integração de módulos e a resolução de dependências técnicas, garantindo que os componentes funcionem de forma coesa. A implementação é acompanhada por atividades de verificação e validação, onde se aplicam diferentes níveis de testes, com o objetivo de identificar defeitos, avaliar comportamentos e assegurar que o software cumpre os requisitos estabelecidos. Por fim, a documentação técnica desempenha um papel transversal a todas estas fases, registando decisões, especificações, arquiteturas, APIs e procedimentos de utilização, facilitando a manutenção futura e promovendo uma compreensão consistente do sistema entre programadores e *stakeholders*.

Para além das etapas que compõem o desenvolvimento de software, importa considerar também as metodologias que organizam estas atividades ao longo do ciclo de vida. Os processos tradicionais, como o modelo em cascata (Waterfall)[26], estruturavam o desenvolvimento de forma sequencial, com transições rígidas entre etapas. Com o tempo, esta abordagem revelou limitações em cenários de requisitos dinâmicos, particularmente em contextos de mudança frequente ou incerteza elevada, conduzindo à adoção de metodologias ágeis, que promovem ciclos iterativos, entregas incrementais e adaptação contínua às necessidades do utilizador[15], como ilustrado na Figura 2.1.

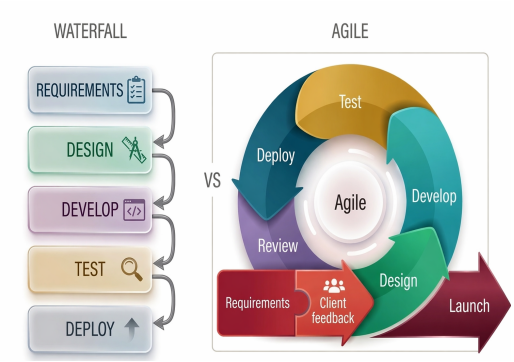

Figura 2.1: Comparação entre os modelos Waterfall e modelos Iterativos

2.1.3 Limitações dos LLMs e Técnicas de Refinamento

Apesar dos avanços dos LLMs, estes modelos apresentam limitações relevantes, como a geração de código incorreto ou incompleto, inconsistências lógicas, alucinações factuais e dificuldades na interpretação de requisitos ambíguos. Estas limitações decorrem, em grande parte, da natureza estatística dos modelos e da ausência de mecanismos internos de verificação semântica.
Uma abordagem para mitigar estas limitações consiste no *fine-tuning* com dados específicos de domínio, com o objetivo de ajustar o comportamento do modelo às necessidades de uma organização ou tarefa concreta. No entanto, esta estratégia implica custos elevados, sobretudo no caso de modelos de grande escala. Cottier et al.[5] demonstram que os custos computacionais associados ao treino e re-treino de LLMs têm aumentado de forma acentuada, com um crescimento médio de 2.4

vezes por ano desde 2016.

Neste contexto, a engenharia de prompts (*prompt engineering*) tornou-se uma alternativa particularmente relevante, fornecendo técnicas que procuram otimizar as instruções fornecidas ao modelo, estruturando-as de modo a orientar o comportamento do LLM sem necessidade de treino adicional. Estudos recentes têm demonstrado que a forma como o prompt é construído influencia significativamente a qualidade do código gerado, a clareza das explicações e a capacidade de o modelo seguir passos complexos[22]. Ao estabelecer padrões, estratégias e boas práticas de formulação de prompts[21], é possível alcançar resultados mais fiáveis e reduzir ambiguidades inerentes à interpretação do modelo. Para além destas boas práticas gerais, a literatura identifica ainda um conjunto variado de técnicas específicas, cada uma concebida para atingir objetivos distintos, desde a melhoria do raciocínio e lógica do modelo, até à redução de alucinações, conforme sistematizado por Sahoo et al.[27].

Dado que esta dissertação visa explorar a automatização de diferentes etapas do desenvolvimento de software, a engenharia de prompts assume um papel central enquanto mecanismo de controlo e refinamento do comportamento dos modelos utilizados. O baixo custo operacional e a ausência de necessidade de treino especializado tornam esta abordagem particularmente adequada ao contexto estudado, permitindo adaptar modelos generalistas às tarefas específicas que compõem a pipeline de geração de software.

2.1.4 Ferramentas de Orquestração de Fluxos

No contexto deste trabalho, a automatização de etapas do processo de desenvolvimento de software implica a coordenação de múltiplos modelos de linguagem, cada um responsável por uma fase distinta da pipeline. Para suportar esta coordenação de forma estruturada e controlada, torna-se necessário recorrer a ferramentas de orquestração que abstraiam a complexidade associada à gestão dos fluxos de geração, ao encadeamento de chamadas a diferentes modelos e à integração de mecanismos de validação e refinamento iterativo.

Estas plataformas funcionam como camada de controlo central da pipeline, permitindo estruturar o processo em componentes independentes e reutilizáveis. Cada componente pode encapsular uma tarefa específica, como a invocação de um modelo, o processamento de uma resposta ou a aplicação de uma regra de validação, sendo depois interligado com os restantes de forma a produzir um fluxo coerente e rastreável. Esta natureza modular facilita não só o desenvolvimento iterativo da solução, como também a sua manutenção e evolução ao longo do tempo.

Uma característica comum a estas plataformas é a sua capacidade de integração com múltiplos fornecedores de modelos de linguagem, o que permite selecionar o modelo mais adequado para cada tarefa específica sem alterar a arquitetura global da pipeline. No panorama atual, o Dify[1], o

[1] https://dify.ai

LangFlow[2], o Flowise[3] e o n8n[4] são exemplos representativos destas plataformas. Todas disponibilizam ambientes visuais e programáticos para a composição modular de pipelines baseadas em LLMs, suportam integração com múltiplos fornecedores de modelos e permitem incorporar lógica personalizada e mecanismos de validação ao longo do fluxo de geração.

2.2 Trabalho Relacionado

A presente secção reúne o estado da arte relevante para esta tese, apresentando trabalhos, modelos e ferramentas que abordam problemas relacionados com a aplicação de métodos de inteligência artificial ao desenvolvimento de software e a utilização de técnicas de prompting.

2.2.1 Geração Automática de Código

Nesta secção são analisados trabalhos que ilustram várias abordagens para automatizar a transformação de descrições diretamente em código.

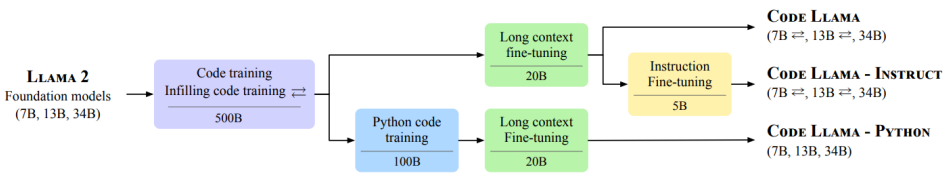

Figura 2.2: Pipeline de especialização do Code Llama[9].

Hoje em dia já existem vários modelos generativos com a finalidade de desenvolvimento de código, ou seja, modelos que foram construídos com o objetivo principal de gerar como resposta um excerto de código que cumpre os requisitos solicitados pelo ser humano. Codex[11], uma variante do GPT-3[13] especializada em código, foi um dos primeiros modelos amplamente conhecidos para transformar descrições textuais em código executável, suportando múltiplas linguagens e tarefas como geração de funções, preenchimento e tradução entre linguagens. Este modelo serviu ainda de base ao GitHub Copilot (que será abordado mais adiante), estabelecendo um marco na utilização prática de LLMs para auxiliar o desenvolvimento de software.
Dando seguimento a esta evolução, surgiram modelos especializados na geração de código, como o CodeLlama [9], uma família de modelos baseada no Llama2 [10], orientada para tarefas como geração, explicação e completamento de código. Estes modelos incluem variantes adaptadas a diferentes contextos de utilização, nomeadamente para suporte a linguagens específicas ou para melhor seguimento de instruções.

[2]https://www.langflow.org
[3]https://flowiseai.com
[4]https://n8n.io

Tal como outros modelos da mesma geração, o CodeLlama foi treinado em grandes volumes de código e linguagem natural, permitindo lidar com sintaxe estruturada e múltiplos paradigmas de programação. A Figura 2.2 ilustra o processo de especialização destes modelos.

Adicionalmente, o StarCoder[12] e a sua evolução, o StarCoder2[8], foram treinados em larga escala no dataset The Stack, destacando-se pelo suporte multilinguagem, janelas de contexto alargadas e mecanismos de treino orientados para segurança e transparência. Estas características tornam-nos referências relevantes para compreender boas práticas na construção de LLMs orientados para código e na conversão fiável de descrições em implementações executáveis.

Para além dos modelos dedicados à geração de código, têm surgido ferramentas práticas de assistência ao programador integradas diretamente nos IDEs (ambientes de desenvolvimento integrado). Um exemplo marcante é o GitHub Copilot[5], inicialmente alimentado pelo modelo Codex. Com a evolução da ferramenta, o Copilot passou também a incorporar modelos mais avançados, como o GPT-4 e GPT-4o. Hoje em dia já inclui modos autónomos como o modo de agente e o agente de programação, capazes de executar ações dentro da IDE ou desenvolver tarefas completas de forma assistida. Ziegler et al.[35] estudaram empiricamente a ferramenta analisando o comportamento da mesma e o seu impacto no processo de desenvolvimento, evidenciando como o preenchimento incremental pode acelerar tarefas de rotina e reduzir o esforço cognitivo do programador.

De forma semelhante, a Amazon apresentou o CodeWhisperer[6], um assistente de programação concebido para gerar sugestões de código contextualizadas em múltiplas linguagens. Yetistiren et al.[33] compararam o CodeWhisperer com outras ferramentas, incluindo o Copilot e o ChatGPT, avaliando a qualidade, correção e segurança das sugestões produzidas. Tal como o Copilot, o CodeWhisperer opera principalmente como um mecanismo de preenchimento inteligente dentro da IDE, contribuindo para automatizar partes do fluxo de escrita de código e acelerar tarefas repetitivas.

A integração crescente destas ferramentas nos IDEs evidencia a adoção prática dos LLMs no desenvolvimento de software, contribuindo para acelerar tarefas rotineiras e aumentar a produtividade dos programadores[24].

Recentemente, surgiram também ferramentas que suportam *Spec-Driven Development* (SDD). A ideia chave do SDD é, em vez de se implementar o código primeiro e documentá-lo posteriormente, começa-se por definir as especificações que irão servir de guia para os agentes de IA. Estas especificações assumem-se como a fonte central de orientação do processo, podendo incluir listas de requisitos, componentes, tarefas e decisões tecnológicas que, em abordagens tradicionais, estariam distribuídas por diversos artefactos de análise e planeamento. Assim, uma parte significativa das atividades que antecedem a implementação é formalizada diretamente nestes ficheiros. Entre

[5] https://github.com/features/copilot
[6] https://aws.amazon.com/pt/q/developer/

as ferramentas que suportam o SDD destaca-se o Spec-Kit[7], desenvolvido pela equipa de IA do GitHub, uma ferramenta que fornece uma interface de linha de comandos para criar e organizar especificações, planos e tarefas capazes de guiar modelos generativos ao longo de um fluxo de desenvolvimento. De forma complementar, o Kiro[8], desenvolvido pela Kiro Labs, apresenta-se como um ambiente integrado que combina edição de especificações com agentes capazes de gerar e atualizar código com base nesses ficheiros estruturados. Embora ainda recentes, ambas as ferramentas ilustram uma nova tendência no desenvolvimento de software onde se criam fluxos automáticos que partem de especificações estruturadas para produzir componentes coerentes.
Além destas ferramentas, têm surgido também propostas que abordam este problema recorrendo a especificações formais. Patil et al.[25] apresentam o spec2code, um framework que combina modelos de linguagem com verificadores formais para gerar código C a partir de especificações estruturadas. Existe também investigação centrada na geração automática das próprias especificações formais. Um exemplo recente é o SpecGen, proposto por Ma et al.[20], que utiliza modelos de linguagem para produzir contratos formais, a partir de código-fonte ou descrições textuais. O sistema gera especificações em linguagens como JML[1] e avalia a sua consistência e verificabilidade utilizando verificadores formais. O SpecGen demonstra o papel crescente dos LLMs na automatização de etapas tradicionalmente manuais.

A Figura 2.3 apresenta uma linha temporal dos sistemas analisados. Importa salientar que o ecossistema atual de ferramentas e modelos generativos para programação é vasto e em rápida evolução, com contributos de organizações como OpenAI, Meta, Google, AWS, Mistral AI e da comunidade de código aberto.

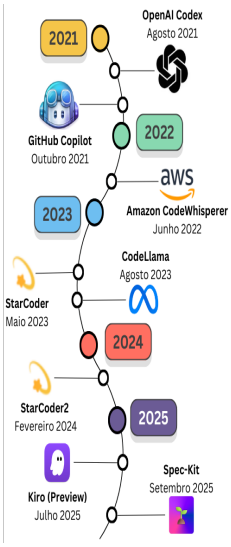

Figura 2.3: Linha temporal de alguns dos modelos e ferramentas de geração de código analisados nesta secção.

[7]https://github.com/github/spec-kit
[8]https://kiro.dev

2.2.2 Engenharia de Prompts

Como já foi referido na secção 2.1.3, a forma como as instruções são fornecidas a um modelo de linguagem tem um impacto direto na qualidade do código que o mesmo consegue gerar. Diversos trabalhos mostram que técnicas de engenharia de prompts podem melhorar significativamente o desempenho em tarefas de linguagem natural para código [19, 17, 6]. No resto da secção, são apresentados estudos que aplicam estas técnicas em cenários reais de geração de código, evidenciando a sua importância prática no desenvolvimento de sistemas baseados em LLMs.

Um dos estudos no contexto da engenharia de prompts aplicada à geração de código é o trabalho de Liu et al.[19], que investiga a forma como diferentes estratégias de prompting influenciam o desempenho do ChatGPT em tarefas de geração de código. Os autores avaliam o modelo no benchmark CodeXGlue, um conjunto de tarefas padronizadas para programação que inclui geração, tradução, refatoração e preenchimento de código, e demonstram que a formulação do prompt tem um impacto substancial na qualidade do código gerado. O estudo aplica técnicas como *chain-of-thought*, instruções comportamentais e otimizações multi-etapa, mostrando que o desempenho do ChatGPT pode melhorar significativamente. Esta conclusão foi alcançada através de métricas como o BLEU que avalia a semelhança entre o código gerado e a solução de referência através da contagem de n-gramas, isto é, sequências contíguas de n tokens que permitem medir a sobreposição entre as duas versões de código e o CodeBLEU, que amplia a abordagem anterior ao incluir informação específica de programas, analisando não apenas n-gramas mas também a estrutura sintática representada por árvores AST e as relações de fluxo de dados.

Adicionalmente Wang et al.[32], afirmam que, apesar dos avanços recentes nos LLMs, continua a ser difícil garantir que estes produzam código correto e robusto de forma consistente. Para mitigar este problema, os autores propõem o PET-Select, um método que seleciona automaticamente a técnica de prompting mais adequada com base na complexidade do problema. Avaliado em benchmarks como MBPP e HumanEval, o PET-Select mostrou melhorias moderadas na qualidade do código e reduções significativas no custo de geração, demonstrando que a escolha informada da estratégia de prompting pode ter impacto real no desempenho de modelos orientados à programação.

Um outro contributo é apresentado por Khojah et al.[17], que exploraram o impacto de diferentes técnicas de prompt engineering na geração de código. Os autores introduzem o CodePromptEval, um dataset composto por 7072 prompts concebido para avaliar cinco técnicas distintas (few-shot, persona, chain-of-thought, function signature e list of packages) aplicadas à geração de funções completas em 3 modelos, o GPT-4o, Llama 3 e Mistral. O estudo mostra que algumas destas técnicas influenciam de forma significativa a correção e qualidade do código produzido. Contudo, a combinação de várias técnicas não garante necessariamente melhorias adicionais.

Para além das técnicas de prompting aplicadas diretamente ao enunciado da tarefa, existe uma

linha de investigação que explora estratégias de closed-loop prompting, onde o modelo é instruído a analisar e refinar o próprio código após a primeira geração. Ding et al.[6] apresentam o CYCLE, um método que combina geração inicial com iterações sucessivas de auto-refinamento orientadas por feedback, mostrando melhorias na qualidade final do código. De forma complementar, Zhou et al.[34] propõem o RefineCoder, que incorpora um mecanismo adaptativo de crítica e correção para permitir que o LLM identifique e ajuste erros no código que produz. Ambos os trabalhos reforçam que a melhoria iterativa guiada pelo próprio modelo pode ser uma alternativa eficaz às abordagens que dependem exclusivamente do prompt inicial, contribuindo para aumentar a fiabilidade do código gerado.

Capítulo 3

Visão Geral e Abordagem à Solução

O presente capítulo descreve a solução proposta nesta dissertação, apresentando a sua arquitetura e os princípios que orientam o seu desenvolvimento. Com base nas limitações identificadas no estado da arte relativamente à aplicação de modelos de linguagem no desenvolvimento de software de forma integrada, define-se uma abordagem estruturada que visa explorar o seu potencial de forma controlada e consistente.

A solução proposta assenta numa pipeline modular, composta por diferentes módulos responsáveis pela geração e transformação de artefactos ao longo do ciclo de vida do desenvolvimento de software. Esta organização permite decompor o processo em etapas distintas, facilitando o controlo, a validação e a evolução independente de cada componente.

Para além da definição da arquitetura global, são também apresentadas as fases que orientaram o desenvolvimento da solução, bem como os principais desafios e decisões de design que influenciaram a sua construção.

3.1 Arquitetura e Design da Solução

3.2 Fases de Desenvolvimento

3.3 Desafios e Decisões de Implementação

Capítulo 4

Desenvolvimento Técnico da Solução

4.1 Desenvolvimento Inicial com Prompts Manuais

4.2 Fluxo de Automação em n8n

4.3 Iterações e Refinamentos

Capítulo 5

Testes e Metodologia de Avaliação

5.1 Plano Experimental

5.2 Métricas e Critérios de Avaliação

Capítulo 6

Resultados e Discussão

6.1 Análise Comparativa

6.2 Discussão dos Resultados

Capítulo 7

Conclusão

7.1 Conclusões Finais

7.2 Trabalho Futuro

Apêndice A

Material de Apoio

A.1 Prompts Utilizados

A.2 Resultados Detalhados

Bibliografia

[1] Java modeling language (jml). `https://www.cs.ucf.edu/~leavens/JML/index.shtml`. Accessed: 2025-11-22.

[2] Joan Bolker. *Writing Your Dissertation in Fifteen Minutes a Day*. Hachette Books, 2008.

[3] E. M. Bouzian. Use case and uml scenarios engineering for building b2b e-commerce. *International Journal of Advanced Trends in Computer Science and Engineering*, 9(4):5428–5432, 2020.

[4] Marco Brambilla, Jordi Cabot, and Manuel Wimmer. *Model-Driven Software Engineering in Practice*. Morgan and Claypool, 2012.

[5] Ben Cottier, Robi Rahman, Loredana Fattorini, Nestor Maslej, and David Owen. The rising costs of training frontier ai models. *arXiv preprint arXiv:2405.21015*, 2024.

[6] Yangruibo Ding, Marcus J. Min, Gail E. Kaiser, and Baishakhi Ray. Cycle: Learning to self-refine the code generation. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1):392–418, 2024.

[7] Umberto Eco. *How to Write a Thesis*. The MIT Press, 2015.

[8] Anton Lozhkov et al. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024.

[9] Baptiste Roziere et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[10] Hugo Touvron et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[11] Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[12] Raymond Li et al. Starcoder: May the source be with you. *Transactions on Machine Learning Research*, 2023.

[13] Tom B. Brown et al. Language models are few-shot learners. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33*, 2020.

[14] David Evans, Paul Gruba, and Justin Zobel. *How to Write a Better Thesis*. Springer, 2014.

[15] Marta Fernandez-Diego, Erwin R. Mendez, Fernando Gonzalez-Ladron de Guevara, Silvia Abrahao, and Emilio Insfran. An update on effort estimation in agile software development: A systematic literature review. *IEEE Access*, 8:166768–166800, 2020.

[16] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8), 2024.

[17] Ranim Khojah, Francisco Gomes de Oliveira Neto, Mazen Mohamad, and Philipp Leitner. The impact of prompt programming on function-level code generation. *IEEE Transactions on Software Engineering*, 51(8):2381–2395, 2025.

[18] Sachiko Lim, Aron Henriksson, and Jelena Zdravkovic. Data-driven requirements elicitation: A systematic literature review. *SN Computer Science*, 2(1):16, 2021.

[19] Chao Liu, Xuanlin Bao, Hongyu Zhang, Neng Zhang, Haibo Hu, Xiaohong Zhang, and Meng Yan. Improving chatgpt prompt for code generation. *arXiv preprint arXiv:2305.08360*, 2023.

[20] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. Specgen: Automated generation of formal program specifications via large language models. In *ICSE*, pages 16–28, 2025.

[21] Ggaliwango Marvin, Hellen Nakayiza, Daudi Jjingo, and Joyce Nakatumba-Nabende. Prompt engineering in large language models. In *International Conference on Data Intelligence and Cognitive Informatics*, pages 387–402, 2023.

[22] Antonio Mastropaolo, Luca Pascarella, Emanuela Guglielmi, Matteo Ciniselli, Simone Scalabrino, Rocco Oliveto, and Gabriele Bavota. On the robustness of code generation techniques: An empirical study on github copilot. In *45th IEEE/ACM International Conference on Software Engineering (ICSE 2023)*, pages 2149–2160. IEEE, 2023.

[23] Marjan Mernik, Jan Heering, and Anthony M. Sloane. When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4):316–344, 2005.

[24] Nettur, Suresh Babu, Karpurapu, Shanthi, Nettur, Unnati, Gajja, Likhit Sagar, Myneni, Sravanthy, Dusi, and Akhil. The role of github copilot on software development: A perspective on productivity, security, best practices and future directions. *arXiv preprint arXiv:2502.13199*, 2025.

[25] Minal Suresh Patil, Gustav Ung, and Mattias Nyberg. Towards specification-driven llm-based generation of embedded automotive software. In *AISoLA*, pages 125–144, 2024.

[26] W. W. Royce. Managing the development of large software systems: Concepts and techniques. In *Proceedings of ICSE*, pages 328–339, 1987.

[27] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*, 2024.

[28] Douglas C. Schmidt. Guest editor's introduction: Model-driven engineering. *Computer*, 39(2):25–31, 2006.

[29] Paul J. Silvia. *How to Write a Lot: A Practical Guide to Productive Academic Writing*. APA LifeTools, 2018.

[30] Ian Sommerville. *Software Engineering*. Pearson, 10 edition, 2015.

[31] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

[32] Chung-Yu Wang, Alireza DaghighFarsoodeh, and Hung Viet Pham. Selection of prompt engineering techniques for code generation through predicting code complexity. *arXiv preprint arXiv:2409.16416*, 2024.

[33] Burak Yetistiren, Isik Ozsoy, Miray Ayerdem, and Eray Tuzun. Evaluating the code quality of ai-assisted code generation tools: An empirical study on github copilot, amazon codewhisperer, and chatgpt. *arXiv preprint arXiv:2304.10778*, 2023.

[34] Changzhi Zhou, Xinyu Zhang, Dandan Song, Xiancai Chen, Wanli Gu, Huipeng Ma, Yuhang Tian, Mengdi Zhang, and Linmei Hu. Refinecoder: Iterative improving of large language models via adaptive critique refinement for code generation. arXiv:2502.09183, 2025.

[35] Albert Ziegler, Eirini Kalliamvakou, Shawn Simister, Ganesh Sittampalam, X. Alice Li, Andrew Rice, Devon Rifkin, and Edward Aftandilian. Productivity assessment of neural code completion. *arXiv preprint arXiv:2205.06537*, 2022.